Array 30 (2026) 100979

Contents lists available at ScienceDirect

Array

journal homepage: www.sciencedirect.com/journal/array

# A comparative evaluation of ChatGPT-4o and DeepSeek-R1 for AI-generated code quality using software engineering metrics

Sally Almanasra [a], Khaled Suwais [b,*], Suad AlRamouni [a]

[a] College of Computer and Information Sciences, Prince Sultan University, Riyadh, Saudi Arabia
[b] Faculty of Computer Studies, Arab Open University, Riyadh, Saudi Arabia

ARTICLE INFO

*Keywords:*
Artificial intelligence
Software engineering
Code quality
ChatGPT
DeepSeek

ABSTRACT

The paper presents an empirical evaluation of code quality generated by two of the most popular large language models, ChatGPT-4o and DeepSeek-R1, as an effort to evaluate their capabilities and weaknesses in real-world software development contexts. The study was conducted as a large-scale study consisting of an experiment that tested 600 problems in Java and Python programming, and the problems were chosen as those covering a wide range of software engineering problems. Each solution produced by AI was evaluated based on five primary components of the quality of the code: code structure and organization, method and class design, naming conventions, error handling and validation, and commenting and documentation. The choice of these metrics was to count not only the structural strength of code, but also the semantic one, when introducing an objective picture of code quality. The outcome indicates that DeepSeek-R1 is more likely to generate code with stronger maintainability-oriented quality, greater consistency, and better structural organization than ChatGPT-4o. The average scores were superior and the standard deviations were lower, and the failure rates were much lower (in all the quality parameters) in both languages in DeepSeek-R1. ChatGPT-4o scored high on the method design and naming conventions, but was more variable and performed less well on error handling and documentation, which are important to maintainability and robustness. The results indicate that DeepSeek-R1 achieved higher mean scores than ChatGPT-4o in code structure and organization for Python (8.89 vs. 7.49) and Java (8.83 vs. 7.07), method/class design for Python (8.73 vs. 7.11) and Java (8.65 vs. 7.06), error handling and validation for Python (7.15 vs. 5.06) and Java (7.15 vs. 4.69), and documentation/comments for Python (7.24 vs. 5.55) and Java (6.35 vs. 4.53). Functional correctness testing also showed higher full-problem correctness rates for DeepSeek-R1 in Java (77.8% vs. 68.4%) and Python (81.6% vs. 72.9%). ChatGPT-4o remained competitive in naming conventions, particularly in Java (7.44 vs. 6.61).

## 1. Introduction

The emergence of artificial intelligence (AI) has extensively changed the appearance of software development, especially with the introduction of AI-based code generation. It is a recent development with the use of machine learning (ML) and deep learning (DL) systems in automating many parts of the coding process to increase the efficiency of developers and tackle the complexity of software engineering. The use of AI in code generation is no flash in the pan; this is a fundamental change in the process of conceiving, programming, and supporting software.

Automation of work on such tasks as code completion, translation, and synthesis is carried out through the automation of many different methods, including AI-driven code generation includes natural language processing (NLP) and symbolic machine learning. In particular, AI is useful in automating code generation, which highlights ML algorithm abilities in this domain [1]. It also demonstrated that deep reinforcement learning can be used in many ways in improving the code generation processes, which is further evidence that AI can help in optimizing software engineering work [2].

AI still has a revolutionary contribution throughout the software sphere that covers issues related to the classical models of coding and makes the process more efficient [3]. GitHub Copilot and OpenAI Codex are tools built on top of large language models (LLMs) [4] that contextually suggest, simplify workflows, and decrease time-to-market. The progress in this area is assisted by such structured systems as the Six-Tier Architecture of AI-Generated Software Development [5] that

* Corresponding author.
*E-mail address:* khaled.suwais@arabou.edu.sa (K. Suwais).

https://doi.org/10.1016/j.array.2026.100979
Received 4 March 2026; Received in revised form 29 May 2026; Accepted 2 June 2026
Available online 10 June 2026
2590-0056/© 2026 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

methodically incorporates AI throughout the software development life cycle. Nevertheless, some of the challenges can still be noted, especially in terms of reliability, explainability, and accuracy and more research is needed to improve the methodologies of AI and secure its performance in practice. The AI-driven code generation has a bright future. Continuous developments in ML and DL are helping programs accomplish progressively complex automation and move into technologies like industrial automation and web-based process management [6]. This development indicates the rising need for AI technologies in various industries and their possible transformation of the software development process.

ChatGPT-4o shows a fundamental progress in the field of LLMs, especially when it comes to the area of code generation [7]. The current model has been developed with the longstanding competencies of its predecessors, including the GPT-3, by employing a better NLP methodology, a larger training set comprising a wide range of programming languages and coding patterns. This has been achieved with a very powerful tool that can comprehend and write source code based on a language description, effectively making those interactions between a developer and the computer code much more natural. This would be mandatorily required in contemporary software development, where the swiftness to convert ideas to working code can greatly impact the productivity and innovativeness.

ChatGPT-4o architecture is optimized with a large variety of capabilities for handling coding, such as code completion, code translation, and code detection issues. These are essential capabilities because software development is becoming heavily focused on the speed of iteration and release of features. As an example, the research by Taeb et al. addresses this issue of AI code-generating systems being efficient and safe to write with coding in mind; the approach of including security objectives into the code development is mentioned [8]. The capability of ChatGPT-4o to provide contextually relevant snippet code not only makes the entire development process easier and smoother, but also lessens the cognitive load of developers, thus enabling them to think more on a higher level regarding the design and architecture [9].

Besides being capable of coding, ChatGPT-4o comprises enhancements making the tool more useable and effective in education. To the point that Cambaz and Zhang claim that LLMs, such as ChatGPT, could transform programming classes and allow students to get immediate feedback and help [9]. In particular, this feature helps new programmers who might be bad at grasping complicated code theories.

DeepSeek is also an extended language model, but it also has different abilities, such as targeting code generation with the help of deep learning. It employs advanced algorithms in analyzing the existing codebases and determining patterns on which new code could be based. The collaboration of ChatGPT-4o and DeepSeek demonstrates how various AI-powered methods can be used to maximize the software development process. According to Idrisov and Schlippe, when generative AI becomes a part of the coding process, the development becomes not only more efficient but also acquires a broader reach in terms of offering a context of programming knowledge more commercially available to people [10].

In addition, these models can be used in the classification of more than conventional software development. Leung and Murphy emphasize that software engineering with large language models (LLMs) may be one of the most radical steps towards significant progress in the already existing fields, like web creation and mobile apps [7]. The potential to create snippets of code or complete functions based on the use of natural language prompts allows quick prototyping and iterative development, which are essential in the modern high-velocity technology industry. ChatGPT-4o and DeepSeek are the leaders of AI-based code generation, and they are the most powerful tools that contribute to the development flow and can become teaching materials for learners of any skill level.

The goal of this study is to implement a comparative assessment of the quality of the code generated by ChatGPT-4o and DeepSeek-R1 based on key software engineering metrics, with particular attention to positioning the comparison as a general-purpose versus reasoning-oriented model evaluation. Using a set of 600 problems with AI-suggested solutions to Java and Python programming language tasks, the study evaluates the strengths, limitations, and overall performance of each model when generating a large volume of high-quality, maintainable, and well-structured code. In constructing the dataset for this study, we selected 600 programming problems to cover a broad spectrum of real-world software engineering challenges. The collection includes a mix of classical algorithmic coding exercises and practical system design scenarios. By spanning multiple levels of difficulty and complexity, this diverse problem set reflects real-world applications and ensures that the evaluation covers tasks relevant to actual industry contexts. The five major factors of quality assessment entail code structure and organization, method and class design, naming conventions, error handling/validation, and documentation and comments. By comparing the results across these metrics, models, and programming languages, the study identifies performance patterns and limitations while examining whether a reasoning-oriented model demonstrates different quality behavior from a general-purpose model. The results contribute to ongoing discussions about AI-assisted development and provide developers, researchers, and practitioners working with AI generative models with practical insights on how to apply them in working environments.

To systematically assess the capabilities and limitations of ChatGPT-4o and DeepSeek-R1 in AI-driven code generation, this research is guided by three key research questions. (RQ1) How do ChatGPT-4o and DeepSeek-R1 perform in generating high-quality code? This question evaluates the overall effectiveness of both models by analyzing their ability to generate well-structured and maintainable code, and, where deterministic test cases are available, functionally correct code across varying levels of problem complexity. (RQ2) What are the common issues and limitations? This question focuses on identifying recurring flaws such as poor documentation, inconsistent naming conventions, inefficient error handling, and suboptimal class or method design. These issues may limit the real-world applicability of AI-generated code. (RQ3) How does Java code compare to Python in terms of structure, design, and maintainability? Given the fundamental differences in programming paradigms, this question explores whether the models produce significantly different outputs when working with Java versus Python. Answering these research questions will enable a comprehensive comparative analysis of the two AI models, shedding light on their practical suitability for software engineering tasks.

The quality of AI-generated code will become a more pertinent study to the applications of real-world practice due to its relevance in regards to software development effectiveness and reliability. Automation of repetitive jobs through AI code can greatly increase the productivity of the developer and give them time to work on more complicated tasks [11]. Meanwhile, the concept of AI in software development requires adequate consideration because the outcome of this process has to embrace the requirements related to industry standards and users. Since AI technologies are still developing, it is necessary to assess how such technology affects the quality of the code in the development of computing software applications to preserve the trust and utility of such software [12].

With next-generation code generation rapidly advancing regardless of artificial intelligence, new opportunities and new challenges stand to be faced in the world of software engineering. Such models as ChatGPT-4o and DeepSeek-R1 provide the opportunity to accelerate development, minimize human effort, and achieve productivity due to automatically generated codes. Nevertheless, the quality of the code is still a matter of concern. Ineffectively coded or badly formatted AI code can jeopardize maintainability, cause vulnerabilities, and increase the cost of maintenance in the long term. It is important to ensure that such code meets the best practice in the industry so that any technical debt can be minimized and not put the performance of such systems in danger.

The work will give answers to these issues by comparing ChatGPT-4o

and DeepSeek-R1. The paper seeks to offer a comprehensive understanding of the advantages and deficiencies of the tools as it considers them via assessments of the active code generated by such tools based on important software engineering parameters. The results will facilitate developers, researchers, and industry members to make the right choice on whether to use AI in their tasks and make this new tool efficient, safer, and sustainable.

The rest of this paper is organized as follows. Section 2 provides a literature review of the relevant research in AI-aided code generation and approaches to evaluation. Section 3 explains the experiment environment, the dataset construction, selection of the tasks and the evaluation system. In Section 4, the results between Java and Python are presented comparatively with a statistical and graphical study. Section 5 refers to the strengths and weaknesses of each of the models, practical implications, and ways of enhancing the quality of AI-generated code. In section 6, the study limitations and future study directions are stated. Lastly, Section 7 is a conclusion of major findings and their implication for the changing role of AI in software development.

## 2. Related work

Code quality of AI-generated code is also a growing field of research interest, especially with AI becoming more and more a part of software development workflows. One of the major issues that are associated with this field is the code quality and security. Taeb et al. emphasize the necessity to analyze the efficacy of AI code generators to pay attention to the security measures these tools may pose, stating that although they are likely to improve work efficiency, they can also result in vulnerabilities. Negri-Ribalta et al. confirm this anxiety and demonstrate that certain AI implementations generate code containing known vulnerabilities [9] [13]. This twofold potential, positive and negative importance, explains why AI-generated code still needs investigation in terms of security. The developers need to be keen about the risk that they might be causing bugs and security holes that may subvert the integrity of the software [13].

AI is also capable of enhancing the effectiveness of software development since it helps to notice problems that human programmers would not recognize. Alenezi et al. explain that AI will allow them to solve issues early enough and even prevent them, which will result in high-quality decisions [14]. Nevertheless, the success of AI relies mostly on the quality of the training information as well as the algorithms. According to Idrisov and Schlippe, there is a requirement to have strong evaluation tools to compare machine-written code and the code written by a human programmer [10].

Another vital factor that affects the quality of code is the knowledge of AI engineers. Golendukhina et al. indicate that a lack of clear recommendations to AI engineers may lead to the absence of software quality assurance [15]. The problem should be of particular concern due to the adverse effects of the rapid implemented use of AI-generated code in real life. To fix this, there is an increasing demand for education models that train developers to use AI practices in an effective way, while still being able to uphold high software quality.

A systematic literature review provided by Odeh et al. helps to demonstrate the status and issues of AI-driven code generation in a broad sense [1]. As they review, they point out the necessity of continued research to eliminate the present limitation and enhance code quality. Although both ML and DL algorithms have demonstrated some potential in the improvement of generated code quality, there still must be more empirical research to cement such results regarding alternative programming languages and scenarios.

A study conducted by Kharchenko et al. also proves that a well-rounded quality model of AI systems is essential. Their effort supports the assessment of AI-generated code in regards to its maintainability, readability, and respect of industry benchmarks [16]. This kind of end-to-end implementation is necessary to make sure that any code produced by similar models, such as ChatGPT and DeepSeek, would be up to our expectations in terms of current program writing. The concern of AI-generated code assessment has already drawn increased scholarly interest due to the rapid development of LLMs. As the models, like ChatGPT or DeepSeek, gain prominence, it is critical to conduct systematic tests of the model output to develop an understanding of the abilities, weaknesses, and implications concerning real-world software engineering work.

A recent article [17] gives an overview of the possibilities and constraints of ChatGPT-4o as a code generator in a range of programming languages (such as Java and Python). The research will concentrate more on performance-related data like runtime and memory consumption, as well as create a comparison of AI-based and human-based code. The results show that ChatGPT-4o had a high overall success rate of around 90 percent in producing functional code, but performance in different languages is not equal, and Java has higher success rates in comparison with Python. In the research, more attention should be paid to the issue of code maintainability as ChatGPT-4o was able to efficiently remove the unnecessary variables, and, therefore, enhance the efficiency of the code.

Liu et al. performed an in-depth assessment of the snippets of the code generated by ChatGPT and focused on accuracy, clarity, and security [18]. Their findings demonstrate that ChatGPT may generate functional code, but in some scenarios, the result is often unclear, and it can have security problems in the output. These results are consistent with those highlighted by Idrisov and Schlippe, who found that ChatGPT had a successful code generation rate of 22.2% on diverse problems, as well as its performance being quite good on complex coding tasks [10]. The significance of methodical assessment and verification tests to establish the efficacy of AI-generated coded applications in production is highlighted by such evaluations.

The open-source AI by DeepSeek has positioned itself to be competitive in code generation tasks. As the review by Abraham states, DeepSeek models, and the DeepSeek-R1 model in particular, are rising in popularity because of their affordability and performance that match proprietary products, including those produced by OpenAI [19]. DeepSeek can serve as an alternative for developers who want to access high-quality code generation tools but cannot afford the licensing fees of commercial platforms.

An empirical testing of several generative engines by Tosi, such as ChatGPT and DeepSeek, shows that high-quality benchmarking is an essential method of determining code quality [11]. The experiment is done through human-applied test suites to assess the correctness and the quality of the generated code by offering meaningful comparisons between different models. Coignion et al. take this strategy further by presenting a new way of assessing the effectiveness of LLM-based code compared to solutions produced by humans [20]. Their results show that although LLMs can generate code of similar performance, there is still a lot of room to improve reliability and maintainability.

To perform additional improvements, Ding et al. introduce the CYCLE framework that allows LLMs to improve their code generation based on execution and test suite responses, resulting in an iteration process [21]. Such a self-improvement mechanism can increase the reliability of the resulting AI-generated code to make it better to use in production. Equally, Liu et al. discuss the reliability and explainability of code generation driven by automation and place more importance on the necessity to see how the internal process of code generation works [22]. According to their study, the models would become stronger, so the focus is on solving challenges like data replication and input insensitivity.

Contemporary research has also been brought to the vulnerabilities that the code generated by the AI faces regarding security. Jamdade and Liu study the security generation of codes using ChatGPT in pilot research that points out the possible risk under an application in web settings and presents the necessity of involving security features in the process of generating codes [23]. This is of particular concern as cyber-attacks and other risks are becoming increasingly common, and

security culture must be taken seriously when developing software.

A particularly relevant recent study is that of Cotroneo et al. [24], who conducted a large-scale comparison of human-written and AI-generated Python and Java code. Their study analyzed more than 500,000 code samples and categorized defects using Orthogonal Defect Classification, while security weaknesses were mapped using MITRE's Common Weakness Enumeration framework. Their findings showed that AI-generated code tends to be simpler and more repetitive, but may also be more disposed to unused constructs, hardcoded debugging artifacts, and high-risk security vulnerabilities. This work is important as it demonstrates that several weaknesses observed in AI-generated code, including defect patterns and security concerns, have already been empirically documented at scale.

Recent studies have established that AI-generated code often exhibits recurring quality issues, particularly in documentation, exception handling, maintainability, and security. Siddiq et al. analyzed ChatGPT-generated Java and Python code from developer interactions and reported issues such as improper documentation, undefined or unused variables, and weaknesses in exception and resource management. Similarly, Liu et al. systematically examined thousands of ChatGPT-generated Java and Python programs and found that, although many generated solutions were functionally correct, a considerable portion suffered from runtime errors, incorrect outputs, and maintainability issues. These studies demonstrate that the weaknesses observed in AI-generated code are not isolated cases but recurring concerns across different tools and evaluation settings.

Alternative studies show a shift from direct LLM-based code generation toward more structured, software-engineering-aware pipelines. Nguyen et al. [25] demonstrated that compact models can be effectively fine-tuned for code generation using techniques such as LoRA/QLoRA, 4-bit quantization, SFTTrainer, and FlashAttention, with Phi-3-Code outperforming several baselines. Another Nguyen et al. [26] study proposed an automated UML generation and validation framework that generates requirements, converts them into PlantUML diagrams, and validates outputs using vision-language models, achieving high compilation success and strong agreement with expert evaluation. Collectively, these studies emphasize that effective AI-assisted software development depends not only on model capability, but also on structured design artifacts, validation, and alignment with software engineering practices.

Current literature (see Table 1) shows that although AI code generation has been widely studied, there is still a significant gap in language-specific comparative analysis. The present study differs from Cotroneo et al. in three main ways. First, Cotroneo et al. primarily compare human-written code with AI-generated code, whereas the present study focuses on a direct comparison between two newer AI models, ChatGPT-4o and DeepSeek-R1. Second, while Cotroneo et al. ground their analysis in defect classification, security vulnerability mapping, and structural complexity, this study evaluates software engineering quality using five maintainability-oriented dimensions. Third, the distinctive focus of this study is the reasoning-oriented versus general-purpose model comparison. DeepSeek-R1 is treated as a reasoning-oriented model, while ChatGPT-4o represents a general-purpose multimodal model. Therefore, the contribution of this study is not to rediscover known weaknesses in AI-generated code, but to examine whether a reasoning-oriented model produces more consistent and maintainable Java and Python code than a general-purpose model under the same controlled task and evaluation conditions.

Therefore, the contribution of the present study is not focusing only on the identification of documentation or error-handling weaknesses, which have been reported in prior work. Rather, the contribution of this study is to examine whether a reasoning-oriented model produces more consistent and maintainable Java and Python code than a general-purpose model under the same controlled task and evaluation conditions. Unlike studies that focus on a single model, a single language, or a limited set of quality indicators, this work integrates five software engineering quality dimensions and applies descriptive, distributional, correlation, and regression-based analyses. This enables a more detailed understanding of how model behavior differs across languages and quality dimensions. This gap offers an opportunity to examine whether language-specific features influence the correctness, maintainability, and efficiency of code generated by advanced AI models.

Accordingly, the novelty of this study lies in its controlled comparison of ChatGPT-4o and DeepSeek-R1 as representatives of general-purpose and reasoning-oriented AI code generation models, respectively, rather than in the isolated observation that AI-generated code may contain documentation or error-handling weaknesses.

**Table 1**
Summary of related works.

| Reference | Core focus | language-specific analysis | Model-to-model benchmarking | Integrated quality framework | Main limitations |
|---|---|---|---|---|---|
| [1] | Systematic review of automated code generation techniques | No | No | Partial | Synthesizes but doesn't benchmark. |
| [9] | Early survey of AI code generators | No | No | No | No language split; qualitative only |
| [10] | Calls for unified metrics when comparing AI vs human code | Partial | No | No | Lumps Java & Python together; no cross-model test |
| [11] | Benchmarks ChatGPT vs DeepSeek with human test-suites | No | Yes | Partial | Doesn't isolate language effects. |
| [13] | Shows exploitable weaknesses in LLM-generated C code | No | No | Partial | Single language; single metric |
| [14] | AI detection of latent defects dev efficiency | No | No | No | Evaluates detection, not generated code |
| [16] | Quality models for AI systems covering maintainability & readability | No | No | Yes | Conceptual; not applied to LLMs |
| [17] | Evaluation of the capabilities and limitations of ChatGPT-4o in generating code across various programming languages | Yes | No | No | Only one model; metrics narrow |
| [18] | Evaluates ChatGPT snippets | Partial | No | Partial | No comparative model; limited to snippet-level. |
| [20] | Proposes metric to compare LLM vs human code efficiency | No | Partial | Yes | Evaluated only on small tasks |
| [22] | Analyses reliability & explainability | No | No | Yes | No cross-model/cross-language test |
| [23] | Pilot on ChatGPT secure code generation for web apps | No | No | Partial | Focused on web security only |
| [24] | Large-scale comparison of human-written and AI-generated Python/Java code using ODC defect classification and CWE security mapping | Yes | Partial | Yes | Focuses mainly on human-vs-AI comparison; does not specifically compare reasoning-oriented vs. general-purpose models such as DeepSeek-R1 vs. ChatGPT-4o |

## 3. Methodology

### 3.1. Data collection

The current research applies a set of 600 programming tasks that include a mix of traditional algorithmic problems, mathematical computation, and design problems, as well as those requiring their implementation in the real world. Many algorithmic tasks, such as Binary Search, Merge Sort, the Knapsack Problem, and Graph Traversal, are widely recognized and have direct equivalents on popular coding platforms, including LeetCode [27], HackerRank [28], and Codeforces [29]. Mathematical and computational problems, such as Numerical Integration and the Fourier Transform, are commonly used in scientific programming and numerical analysis. Real-world system design problems—including E-commerce platforms, ride-sharing applications, and blockchain-based voting systems—reflect practical software engineering contexts and are more aligned with system design interviews than algorithmic problem-solving exercises. This diverse problem set enables a broad evaluation of code generation performance across multiple areas of software development.

The problems that have been chosen include a variety of difficulty levels to ensure that all coding skills can be tested. The distribution is emulated in standard coding assessment usage: 30% problems are easy, 40% medium, and 30% hard. This combination covers simple programming tasks, data structures and algorithms problems, mathematical problems, realistic applications, performance system designs, and working with AI and machine learning tasks. Simple issues are oriented towards the basic ideas and simple algorithms, which means that it is possible to assess basic coding skills. Medium problems are even more complex than simple ones, as they imply some use of practical applications, and some reasoning on medium levels of algorithms. Advanced skills and expertise are needed to solve hard problems, which encompass multiple implementations of complex systems, integrations of machine learning, fine logic, and algorithmic tasks, providing a solid measure of high-level programming proficiency. Such a balanced and varied set of problems guarantees that the assessment covers a broad range of the actual software engineering scenarios. Table 2 screens the issues that were used in the experiment.

To reduce task-selection bias, inclusion and exclusion criteria are applied during dataset construction. A task was included only if it represents a common programming problem type, provides a sufficiently clear problem statement, and can be implemented in both Java and Python. Tasks are excluded if they are duplicates, depend on language-specific features that prevent fair Java/Python comparison, require unavailable external services or APIs, lack clear expected behavior or implementation requirements. For functional correctness testing, only deterministic tasks with clear input-output behavior are included in the executable test subset, while open-ended system-design and real-world application tasks are retained for maintainability-oriented assessment only.

The dataset consists of 600 unique programming problems. Each problem is used to generate code in both Java and Python for each AI model. Therefore, for each model, the dataset produced 600 Java code samples and 600 Python code samples, resulting in 1200 code samples per model. Across the two models, ChatGPT-4o and DeepSeek-R1, the full evaluation therefore included 2400 generated code samples. This design ensured statistical balance because both models were evaluated on the same 600 unique problems and the same Java/Python language distribution. The 600 tasks are analyzed according to category, difficulty, programming-language implementation, and testability. The dataset includes both deterministic coding problems and open-ended software engineering design tasks. Deterministic tasks include basic programming, data structures and algorithms, and mathematics problems, which are suitable for functional correctness testing using expected input-output behavior. Open-ended tasks include advanced systems and real-world application scenarios, which are more suitable for assessing structure, design, maintainability, and documentation quality as they do not always have a single expected output. This distribution (Table 3) was intended to ensure diversity across both algorithmic code generation and practical software engineering scenarios.

The sample prompt structure shown in Fig. 1 was used to generate Java code for a given problem using ChatGPT-4o and DeepSeek-R1. Fig. 2(a) and (b) show the resulting code generated by ChatGPT-4o and DeepSeek-R1, respectively. To ensure fairness and eliminate prompt-induced variance, the exact same prompt template was used for every problem and applied consistently to both models across all 600 tasks. The code generation was conducted in a zero-shot setting for both ChatGPT-4o and DeepSeek-R1. No few-shot examples, worked solutions, or model-specific prompting strategies were provided. This choice was made to ensure that both models were evaluated under the same task description and prompt structure.

### 3.2. Evaluation criteria

The five evaluation dimensions used in this study are selected to operationalize established software quality concepts. In particular, the dimensions are aligned with ISO/IEC 25010 software product quality characteristics, especially maintainability, reliability, usability, and security [30]. Code structure and organization, method and class design, naming conventions, and documentation/comments are primarily associated with maintainability because they influence modularity, analyzability, modifiability, readability, and code comprehension, as highlighted in Table 4. Error handling and validation are associated with reliability and security because they affect fault tolerance, robustness against invalid inputs, and resilience under unexpected execution conditions. Therefore, the selected dimensions provide a practical code-level interpretation of recognized software quality concerns.

Defect-oriented taxonomies such as Orthogonal Defect Classification

**Table 2**
Categories of tested problems.

| Category | Difficulty | Count |
|---|---|---|
| AI & Machine Learning | Hard | 30 |
| Advanced Systems | Hard | 60 |
| Basic Programming | Easy | 100 |
| Basic Programming | Medium | 60 |
| Data Structures & Algorithms | Easy | 30 |
| Data Structures & Algorithms | Hard | 30 |
| Data Structures & Algorithms | Medium | 60 |
| Mathematics | Easy | 60 |
| Mathematics | Hard | 30 |
| Mathematics | Medium | 60 |
| Real-World Application | Hard | 30 |
| Real-World Application | Medium | 50 |

**Table 3**
Dataset diversity and testability summary.

| Dataset characteristic | Description |
|---|---|
| Total tasks | 600 programming and software engineering tasks |
| Programming languages | Java and Python generated for each task |
| Models evaluated | ChatGPT-4o and DeepSeek-R1 |
| Difficulty levels | Easy, medium, and hard |
| Main task families | Basic programming, data structures and algorithms, mathematics, AI/ML, advanced systems, real-world applications |
| Deterministic tasks | Basic programming, data structures and algorithms, mathematics, and selected AI/ML tasks |
| Open-ended tasks | Advanced systems and real-world application scenarios |
| Functional correctness testing | Applied to tasks with deterministic input-output behavior |
| Static/manual quality assessment | Applied to all generated code samples |
| Diversity purpose | To cover both algorithmic correctness and maintainability-oriented software engineering quality |

Provide a Java code implementation for the following problem:
The set [1, 2, 3, ..., n] contains a total of n! unique permutations.
By listing and labeling all the permutations in order, we get the following sequence for n = 3:
"123", "132", "213", "231", "312", "321"
Given n and k, return the kth permutation sequence.

Example 1:
Input: n = 3, k = 3, Output: "213"

Constraints:
1 <= n <= 9 , 1 <= k <= n!

**Fig. 1.** Prompt given to ChatGPT-4o and DeepSeek-R1.

(ODC) [24] are also relevant to the assessment of AI-generated code, particularly when the goal is to categorize observed failures or defect reports. However, ODC was not used as the primary framework in this study because our analysis focuses on code-level quality attributes produced by AI models, rather than on classifying confirmed defects after failure observation.

In software engineering, evaluating code quality is essential to ensuring the maintainability, efficiency, and robustness of software systems. High-quality code not only improves performance but also simplifies debugging, supports future development, and promotes effective collaboration among team members. The dimensions are evaluated using a variety of software code quality measures that concentrate on essential elements, including code design and structure, method and class design, effective name, error and validation, and comments and documentation. The combination of such factors helps in making the software product readable, stable and of high quality in general.

Code structure and organization are a measure of the good or bad encoding of code, modularization and coherence. Separation of concerns, modular programming, and other well-defined principles that a well-structured code should follow are usually contrasted with each other by the definition that the various functions are packaged in different modules or components. Maintainability increases with proper file, directory and dependencies organization. Besides this, the compliance with the architectural patterns, like model-view-controller or microservices, can greatly enhance the scalability and the long-term sustainability.

Method and class design analyze the effectiveness of the implementation of strategies and classes. The best design is characterized by single responsibility, high cohesion and low coupling principles. Methods are brief, uncomplicated, and with an emphasis on one task. The related properties and behaviors should be encapsulated in the classes so that modularity and reuse can happen. It is also possible to

**Table 4**
Quality dimensions vs. ISO/IEC 25010 quality characteristics.

| Study evaluation dimension | Related ISO/IEC 25010 quality characteristic | Rationale |
|---|---|---|
| Code structure and organization | Maintainability | Reflects modularity, analyzability, and modifiability of generated code. |
| Method and class design | Maintainability | Captures cohesion, coupling, encapsulation, and responsibility allocation. |
| Naming conventions | Usability/ Maintainability | Supports readability, understandability, and developer comprehension. |
| Error handling and validation | Reliability/Security | Supports fault tolerance, input validation, robustness, and prevention of unsafe behavior. |
| Documentation and comments | Maintainability | Supports analyzability, modifiability, knowledge transfer, and future maintenance. |

```java
import java.util.ArrayList;
import java.util.List;
public class Solution {
    public String getPermutation(int n, int k) {
        List<Integer> numbers = new
ArrayList<>();
        for (int i = 1; i <= n; i++) {
            numbers.add(i);
        }
        int[] factorial = new int[n];
        factorial[0] = 1;
        for (int i = 1; i < n; i++) {
            factorial[i] = i * factorial[i - 1];
        }
        k--;
        StringBuilder sb = new StringBuilder();
        for (int i = n; i > 0; i--) {
            int index = k / factorial[i - 1];
            sb.append(numbers.get(index));
            numbers.remove(index);
            k %= factorial[i - 1];
        }
        return sb.toString();
    }
    public static void main(String[] args) {
        Solution solution = new Solution();

System.out.println(solution.getPermutation(3,
3));
     }
}
```

(a). Java Code Generated by ChatGPT-4o

```java
import java.util.ArrayList;
import java.util.List;
public class KthPermutation {
    public static String getPermutation(int n,
int k) {
        // Calculate factorial values up to n
        int[] factorial = new int[n + 1];
        factorial[0] = 1;
        for (int i = 1; i <= n; i++) {
            factorial[i] = factorial[i - 1] * i;
        }
        // Create a list of numbers from 1 to n
        List<Integer> numbers = new
ArrayList<>();
        for (int i = 1; i <= n; i++) {
            numbers.add(i);
        }

        //Decrement k to make it 0-based index
        k--;

        // Build the k-th permutation
        StringBuilder sb = new StringBuilder();
        for (int i = 1; i <= n; i++) {
            int index = k / factorial[n - i];
            sb.append(numbers.get(index));
            numbers.remove(index);
            k -= index * factorial[n - i];
        }

        return sb.toString();
    }

    public static void main(String[] args) {
        int n = 3;
        int k = 3;
        System.out.println(getPermutation(n, k));
// Output: "213"
    }
}
```

(b). Java Code Generated by DeepSeek-R1

**Fig. 2.** Sample Java codes generated by ChatGPT-4o and DeepSeek-R1.

improve the robustness and flexibility of the code through the use of design patterns, e.g., factory pattern, singleton pattern, or observer pattern.

Naming conventions are also very important in the legibility of code and clarity. Names that are consistent, descriptive of the variable, method, classes and modules aid developers in knowing the functionality and purpose of the elements conveniently without over-relying on the use of documentation. Best practice naming schemes are camelCase for variables and methods, PascalCase for classes and snake_case for constants. Meaningful names will also save time on the onboarding process of new developers and avoid situations when the functionality of the code is misunderstood.

The error handling and validation metric measures the aspect of whether the code is deliberately and effectively anticipating and handling possible errors or exceptions. Strong error management incorporates the try-catch blocks, input validation and fall-forward mechanisms so that the software has the stability in handling unexpected scenarios as well. Input validation is particularly significant concerning the prevention of security vulnerabilities like SQL injection or cross-site scripting [31] [32] [33]. The error handling makes the software more reliable and stable, and facilitates a better and safer user experience.

Documentation and Comments are a crucial metric that is required to sustain the long-term quality of code. The kind of documentation that is clear and concise gives the necessary background to the developers who might touch the code in the future. Intrinsic remarks can describe complicated reasoning, outline procedure parameters, and elaborate on the required and provided data. Consistency and readability can further be increased by using some standardized documentation practices, e.g., Javadoc in Java, or docstrings in Python. However, the comments are meant to augment self-explanatory lines of code and not replace them. The idea is to provide commentary that complements code that has been written well, where further explanation is beneficial.

The measure of code quality presented in these five metrics: code structure and organization, method and class design, naming conventions, error handling and validation, and documentation and Comments are broad guidelines to assess the effectiveness, maintainability, and robustness of a software system. When software engineers apply best practices in every region, they can develop code that is practical, scalable, and readable in numerous development frameworks.

### 3.3. Experimental setup

This study utilizes a combination of both automated and manual code analysis tools blended with a hybrid test arrangement to assess code quality. The main objective here is to evaluate every code sample in the context of the five quality factors mentioned above. The research will give a fair and objective account by combining the power of automated assessment and human evaluation.

The available automated tools are SonarQube, Programming Mistake Detector (PMD) [34] and Java, and PyLint [35] and Python. SonarQube is an open-source code scanning as continuous inspection of code quality across a variety of programming languages. It determines the presence of code smells, bug presence, security risks, and coding standard violations and reports the same through a visual dashboard that contains quantitative measurements with improvement suggestions. This qualifies it especially well to measure structural and design-related properties of the code. PMD is a Java-specific static code analyzer. It finds violations of the coding style, possible bugs, and performance problems, focused on the structure of methods and classes, names, and error handling. PyLint is to the Pythoners. It implements PEP8 coding rules, counts code complexity, gives comments about how things are named, whether documentation is good, or not, and how errors are handled.

To support reproducibility, the model access period, static-analysis tool versions, and analysis settings are recorded. The experiments were conducted between November and December 2025. Since hosted LLM providers may update deployed models and may not expose fixed checkpoint identifiers, the study reports the model name, access platform, access period, prompting mode, and generation settings as illustrated in Table 5.

For DeepSeek-R1 outputs, only the final generated code is considered for evaluation. Any intermediate reasoning text, explanation, or <think>-style content, if produced by the model interface, was excluded before static analysis, functional correctness testing, and manual review. The same principle was applied to ChatGPT-4o outputs: only the final code artifact was evaluated, while surrounding explanations or non-code text were removed. Therefore, the evaluation measures the quality of the generated code rather than the quality of the model's reasoning trace or explanatory text.

To ensure interpretability and consistency, each quality dimension was scored on a 0–10 scale. A score of 0 indicates that the code completely fails to satisfy the expected quality criterion or contains severe violations, while a score of 10 indicates strong compliance with the criterion and no substantial issues detected through either automated or manual review. Therefore, the 0–10 value represents the degree of compliance of a generated code sample with a specific software quality dimension, rather than a direct raw output from any single tool.

The conversion of raw static-analysis outputs into the final reported quality scores followed four steps: first, the output of each tool is mapped to the relevant quality dimensions; second, convert the mapped findings into severity-weighted violation values; third, normalize the violation values to a 0–10 scale within each language and dimension; and fourth, the normalized automated score is combined with the manual rubric score to produce the final dimension-level score.

Since SonarQube, PMD, and PyLint generate heterogeneous outputs, their raw results were not directly added. Instead, tool findings were first

**Table 5**
Model and static-analysis environment.

| Component | Version/identifier | Access date or period | Settings/rule profile |
|---|---|---|---|
| ChatGPT-4o | GPT-4o via ChatGPT/API; exact hosted snapshot not publicly exposed | Nov–Dec 2025 | Zero-shot prompting; one response per prompt; default decoding settings unless otherwise specified; final code only retained |
| DeepSeek-R1 | DeepSeek-R1/DeepSeek-R1-0528 via DeepSeek platform/API; API model name reported as `deepseek-reasoner` | Nov–Dec 2025 | Zero-shot prompting; one response per prompt; reasoning/`<think>` content removed before evaluation; final code only retained |
| SonarQube | SonarQube Server 2025.5 | Nov–Dec 2025 | Default "Sonar way" quality profile for Java and Python; code smells, bugs, reliability, maintainability, and security-related rules enabled |
| PMD | PMD 7.19.0 | Dec. 2025 | Java rulesets covering best practices, code style, design, error-prone, and performance-related checks |
| PyLint | PyLint 4.0.4 | Dec. 2025 | Default PyLint configuration; naming, complexity, documentation/docstring, error-handling, and style checks enabled |
| Java runtime | OpenJDK 21 LTS | Nov–Dec 2025 | Used for Java compilation/execution testing |
| Python runtime | Python 3.12.x | Nov–Dec 2025 | Used for Python execution testing |

mapped to the five quality dimensions. For example, complexity warnings, code smells, and organization-related issues contributed to the code structure and organization score; class design, coupling, and method-level warnings contributed to the method and class design score; naming-rule violations contributed to the naming conventions score; exception-handling, validation, and reliability warnings contributed to the error handling and validation score; and missing or weak comments/docstrings contributed to the documentation and comments score. Each set of tool findings was then normalized to a 0–10 automated score, where fewer and less severe violations resulted in a higher score. The normalized automated score for each dimension was computed using a severity-adjusted normalization approach presented in Eq. (1):

$$S_{tool}(d) = 10 \times \left(1 - \frac{V_d}{V_{max,d}}\right) \quad \text{Eq. (1)}$$

where $S_{tool}(d)$ is the normalized tool score for dimension $d$, $V_d$ is the severity-weighted violation value detected for a given code sample in dimension $d$, and $V_{max,d}$ is the maximum severity-weighted violation value observed for the same dimension within the same programming language. Scores were capped within the range 0–10. This normalization was applied separately by language and dimension to reduce the effect of unequal rule distribution across analyzers.

Because the three static analyzers contain uneven rule sets, raw violation counts were not used as direct quality scores. Instead, the automated analysis relied on dimension-level normalization as highlighted in Table 6. For each programming language and each quality dimension, detected findings were grouped by semantic relevance and converted into a severity-adjusted normalized score on a 0–10 scale. This helps in avoiding penalizing dimensions simply because they were associated with a larger number of analyzer rules. Therefore, the automated score reflects the relative severity and frequency of findings within the same language–dimension context, rather than the number of available rules in the analyzer. Because tool rule repositories evolve across versions and differ in granularity, the study reports mapped rule categories rather than treating the absolute number of rules as a scoring unit.

Tool outputs were harmonized by converting all tool-specific findings into dimension-level normalized scores. For Java code, SonarQube and PMD outputs were used, while for Python code, SonarQube and PyLint outputs were used. Because these tools differ in rule coverage and sensitivity, normalization was performed separately for each language and quality dimension. Where more than one tool produced findings relevant to the same dimension, the normalized tool scores were averaged to form the automated score for that dimension. The manual reviewer score was then combined with the automated score to obtain the final dimension score. In this study, equal weight was assigned to automated and manual assessment, with the final dimension score calculated as follows (Eqs. (2)–(4)):

$$S_{final}(d) = 0.5 \times S_{auto}(d) + 0.5 \times S_{manual}(d) \quad \text{Eq.(2)}$$

$$S_{overall} = \frac{1}{5}\sum_{d=1}^{5} S_{final}(d) \quad \text{Eq.(3)}$$

where

$$S_{auto}(d) = \frac{1}{n}\sum_{i=1}^{n} S_{tool_i}(d) \quad \text{Eq.(4)}$$

where $d$ is the quality dimension, $S_{auto}(d)$ is the normalized automated score for that dimension, $S_{manual}(d)$ is the manual rubric score, and $n$ is the number of applicable tools for the programming language. This hybrid scoring approach was used because automated tools are effective in detecting rule violations, code smells, complexity, and naming/documentation issues, while manual review is necessary to assess semantic appropriateness, design intent, readability, and contextual adequacy. Because the overall quality score is calculated as the arithmetic mean of the five final dimension-level scores, each dimension contributes an equal aggregation weight of 0.20 by design. This aggregation weight is part of the scoring framework and should not be interpreted as a regression coefficient.

Manual code review was also incorporated in the study to capture quality aspects that cannot be fully detected by automated tools. Each generated code sample was evaluated across the five quality dimensions using a pre-defined 0–10 rubric. The 0–10 score represents the degree to which the code satisfies a specific quality dimension, where lower scores indicate serious weaknesses or missing practices and higher scores indicate stronger compliance with expected software engineering practices. Reviewers assessed logical organization, method and class design, readability, error handling and validation, and documentation quality. This qualitative review allowed the evaluation to capture contextual aspects such as the meaningfulness of method names, the appropriateness of class responsibilities, and the adequacy of comments or design choices. To ensure consistency, the same rubric criteria were applied to all samples. A score of 5 or below on any dimension was treated as a "failure," indicating that the code did not meet the baseline expectations for maintainable and reliable software in that aspect. Table 7 presents an example of the dimension-level manual scoring rubric.

The experimental procedure involved analyzing each generated code sample using the applicable automated tools and then conducting a manual rubric-based review. For Java code, SonarQube and PMD were used, while for Python code, SonarQube and PyLint were used. The outputs of these tools were first mapped to the five quality dimensions and normalized to a common 0–10 scale. Afterward, reviewers independently assessed each sample using the predefined rubric for the same five dimensions. The normalized automated score and the manual rubric score were then combined to produce the final dimension-level score for

**Table 6**
Mapping of static-analysis findings to the five evaluation dimensions.

| Quality dimension | SonarQube findings | PMD findings for Java | PyLint findings for Python | Normalization basis |
|---|---|---|---|---|
| Code structure and organization | Code smells, duplicated blocks, complexity-related issues, structural maintainability findings | Complexity, avoidable duplication, unnecessary code, structural style[1] issues | Complexity warnings, unused code, structural style warnings | Severity-weighted findings normalized within this dimension and language |
| Method and class design | Method complexity, class-level design smells, maintainability issues | Long methods, excessive class/method complexity, coupling-related warnings | Too many branches/statements, too many arguments, design-related warnings | Severity-weighted findings normalized within this dimension and language |
| Naming conventions | Identifier naming violations and convention-related maintainability findings | Class, method, variable, and constant naming violations | PEP8 naming violations for variables, functions, classes, and constants | Naming-related findings normalized within this dimension and language |
| Error handling and validation | Reliability, exception-handling, input-validation, and security-related findings | Empty catch blocks, avoidable exceptions, improper exception handling, validation-related findings | Broad exception handling, missing exception handling, invalid input-related warnings | Reliability/security findings normalized within this dimension and language |
| Documentation and comments | Missing or weak documentation-related maintainability findings | Missing comments/Javadocs where detected, documentation-related style findings | Missing module, class, function, or method docstrings | Documentation-related findings normalized within this dimension and language |

**Table 7**
Example of manual scoring rubric.

| Score Range | Error Handling & Validation – Qualitative Criteria |
|---|---|
| 0–3 (Poor) | No error handling or input validation present. The code entirely lacks try/catch blocks or checks for invalid inputs, leading to unhandled exceptions or crashes in edge cases. |
| 4–6 (Basic) | Minimal error handling and validation. The code includes a few try/catch blocks or simple checks, but they are basic. Many potential errors remain unaccounted for. |
| 7–9 (Good) | Robust exception handling and thorough input validation. The code anticipates and handles most common error conditions. Edge cases are considered, and error messages or recovery actions are present for most issues. |
| 10 (Excellent) | Production-ready error management and validation. The code demonstrates comprehensive coverage of edge cases with structured exception handling and full input validation. It exhibits resilience against invalid inputs or unexpected states, meeting industry best practices for reliability. |

each code sample. In this study, equal weighting was used, with the final score for each dimension calculated as 50% automated assessment and 50% manual review. This mixed-method approach balances quantitative evidence from static analysis tools with qualitative reviewer judgment, allowing the evaluation to capture both measurable rule violations and contextual software engineering qualities such as design appropriateness, readability, and documentation usefulness.

The manual review was conducted by three reviewers with backgrounds in software engineering, programming, and code quality assessment. All reviewers had experience in Java and Python programming and were familiar with software engineering principles such as modularity, cohesion, coupling, naming conventions, error handling, and documentation practices. To reduce bias, generated code samples were anonymized before review, and reviewers were not informed whether a given solution was generated by ChatGPT-4o or DeepSeek-R1. Each reviewer independently scored the assigned samples using the predefined 0–10 rubric for each quality dimension. After independent scoring, disagreements were examined and resolved through consensus discussion. When disagreement remained, the final score was determined by the average of the reviewers' scores.

### 3.4. Functional correctness testing

In addition to static and rubric-based code quality assessment, functional correctness testing is conducted for tasks with deterministic inputs and expected outputs. This step is added to ensure that code is not assumed to be correct without execution-based validation. For algorithmic, mathematical, and data-structure problems, test cases are obtained from the original problem statements where available, or derived from the stated examples, constraints, and edge cases. Each generated Java and Python solution is compiled or executed using a controlled testing environment. Java solutions are evaluated using a Java execution harness, while Python solutions are evaluated using a Python test harness.

Three correctness indicators are recorded (Eqs. (4)–(6)): compilation/execution success, test-case pass rate, and full problem correctness. Compilation/execution success indicates whether the generated solution could run without syntax or runtime setup errors. Test-case pass rate represents the percentage of individual test cases passed by the generated solution. Full problem correctness indicates whether the solution passed all test cases assigned to that problem.

$$CSR = \frac{Number\ of\ solutions\ the\ compile\ or\ execute\ successfully}{Total\ number\ of\ executable\ solution} \quad \text{Eq. 4a}$$

$$TCPR = \frac{Number\ of\ passed\ test\ cases}{Total\ number\ of\ test\ cases} \quad \text{Eq. 5}$$

$$PCR = \frac{\text{Number of problems for which all test cases passed}}{Total\ number\ of\ executable\ problems} \quad \text{Eq. 6}$$

Where CSR is the compilation/execution success rate, TCPR is the test-case pass rate, and PCR is the problem correctness rate.

Because the study generated one solution per model for each problem, the correctness evaluation corresponds to a Pass@1-style setting rather than a full Pass@K evaluation. Pass@K is appropriate when multiple independent solutions are generated for the same problem and the evaluation measures whether at least one of the K candidates passes the test suite. In contrast, this study used a controlled single-generation design to ensure that ChatGPT-4o and DeepSeek-R1 are compared under the same prompt and output conditions. Therefore, full problem correctness in this study is interpreted as the model's single-attempt correctness for each task.

Functional correctness is analyzed separately from the five static quality dimensions. This separation prevents structurally clean but functionally incorrect code from being interpreted as production-ready. Accordingly, the term "production-grade" in this study is used to demonstrates stronger maintainability-oriented characteristics and would still require correctness testing, integration testing, and security review before production use. Table 8 represents the functional correctness testing scope.

## 4. Results and analysis

### 4.1. Descriptive statistical analysis

The baseline measurement of the AI-code shows decent variances in performance between ChatGPT-4o and DeepSeek-R1, especially regarding core software engineering metrics. The method of evaluation involves the mean scores and the standard deviation of the two models, giving a reflection of the consistency and effectiveness of writing the two codes in Python and Java. As shown in Tables 9 and 10, DeepSeek-R1 outperforms ChatGPT-4o in code structure and structuring.

In the case of Python, DeepSeek-R1 gives a mean score that is much better than the ChatGPT-4o. The average score of DeepSeek-R1 in Java is 8.83, as compared to 7.07 of ChatGPT-4o. As well as providing better scores, DeepSeek-R1 also has smaller standard deviations in both languages, which is a sign of stronger and more consistent structural quality. These findings indicate that DeepSeek-R1 creates code with a more logical flow and more logical modularity, both of which are essential to code maintainability and scalability.

The approach and class-design metric also address the idea of the superiority of DeepSeek-R1. In the case of Python, it scores an average of 8.73, overtaking ChatGPT-4o by about 1.62 grades. At Java, DeepSeek-R1 is also in the lead with a mean of 8.65 compared to that of ChatGPT-4o at 7.06. The results are evidence that DeepSeek-R1 has a better

**Table 8**
Functional correctness testing scope.

| Task category | Functional correctness testing | Basis for testing |
|---|---|---|
| Basic Programming | Yes | Deterministic inputs/outputs and problem examples |
| Data Structures & Algorithms | Yes | Standard test cases and edge cases |
| Mathematics | Yes | Expected numerical/computational outputs |
| AI & Machine Learning | Partial | Tested only where deterministic expected outputs were available |
| Advanced Systems | Partial/No | Mostly open-ended design tasks without a single expected output |
| Real-World Application | Partial/No | Evaluated mainly through design and maintainability criteria |

**Table 9**
Quality statistics for Python and Java codes generated by ChatGPT-4o.

| | | Code Structure & Organization | Method & Class Design | Naming Conventions | Error Handling & Validation | Documentation & Comments |
|---|---|---|---|---|---|---|
| Python | Mean | 7.49 | 7.11 | 7.12 | 5.06 | 5.55 |
| | STD | 0.61 | 1.12 | 1.40 | 1.22 | 1.71 |
| Java | Mean | 7.07 | 7.06 | 7.44 | 4.69 | 4.53 |
| | STD | 1.01 | 1.25 | 1.59 | 1.04 | 1.76 |

**Table 10**
Quality statistics for Python and Java codes generated by DeepSeek-R1.

| | | Code Structure & Organization | Method & Class Design | Naming Conventions | Error Handling & Validation | Documentation & Comments |
|---|---|---|---|---|---|---|
| Python | Mean | 8.89 | 8.73 | 6.71 | 7.15 | 7.24 |
| | STD | 0.31 | 1.15 | 1.54 | 0.85 | 1.42 |
| Java | Mean | 8.83 | 8.65 | 6.61 | 7.15 | 6.35 |
| | STD | 0.40 | 0.82 | 1.34 | 0.83 | 1.23 |

understanding of the best algorithms of object-oriented design features like single responsibility and encapsulation, so their classes and the construction of the methods are more logically aligned.

ChatGPT-4o has the best mean score of 7.44 (Java) in the category of naming conventions, which is significant because it is above DeepSeek-R1s′ mean score of 6.61. With ChatGPT-4o achieving 7.12 in Python, very slightly above 6.71 with DeepSeek-R1. It means that ChatGPT-4o developed identifiers that appear to be more semantically meaningful and more readable, which helps to make the code more understandable. However, the higher standard deviation in ChatGPT-4o′s outputs reflects greater variability in naming consistency.

Another area that shows a big performance variance is in error handling and validation. The DeepSeek-R1 shows a consistent performance better in both languages, with 7.15 in Python and Java as compared to the ChatGPT-4o, which has significantly lower mean scores of 5.06 and 4.69 in Python and Java, respectively. These findings reveal that DeepSeek-R1 is superior in predicting edge cases and exception handling constructs. As important as validation is in production settings, it is a metric that is particularly applicable in safety- and reliability-critical applications.

DeepSeek-R1, once again, makes ChatGPT-4o in second place in terms of documentation and comments. In Python, it obtains the mean score of 7.24 compared to that of ChatGPT-4o (5.55). In the case of Java, the difference is minimal since DeepSeek-R1 has a score of 6.35 and ChatGPT-4o with 4.53. The findings indicate that DeepSeek-R1 is more consistent and informative with the usage of inline and block comments, which are needed for collaborative development and maintainability in the long term.

### 4.2. Score distribution and variability analysis

The visual analysis of quality score distributions, shown in Figs. 3 and 4, offers deeper insight into how each AI model performs across key software engineering criteria. These distributions highlight both the central tendencies and the variability of the models' outputs—factors that are critical when assessing the robustness and reliability of AI-generated code.

In terms of code structure and organization, DeepSeek-R1 exhibits a tightly clustered and consistently high-scoring distribution for both Python and Java, with scores concentrated around 8 to 9. This suggests that DeepSeek-R1 consistently produces structurally sound code across languages. By contrast, ChatGPT-4o shows a wider spread, especially in Java, where scores range from 4 to 9. This indicates occasional weaknesses in structural design. While ChatGPT-4o′s Python scores are more concentrated around 7 to 8, they still demonstrate greater variability than DeepSeek-R1, reflecting less consistency in maintaining structural quality.

For method and class design, DeepSeek-R1 again displays a more

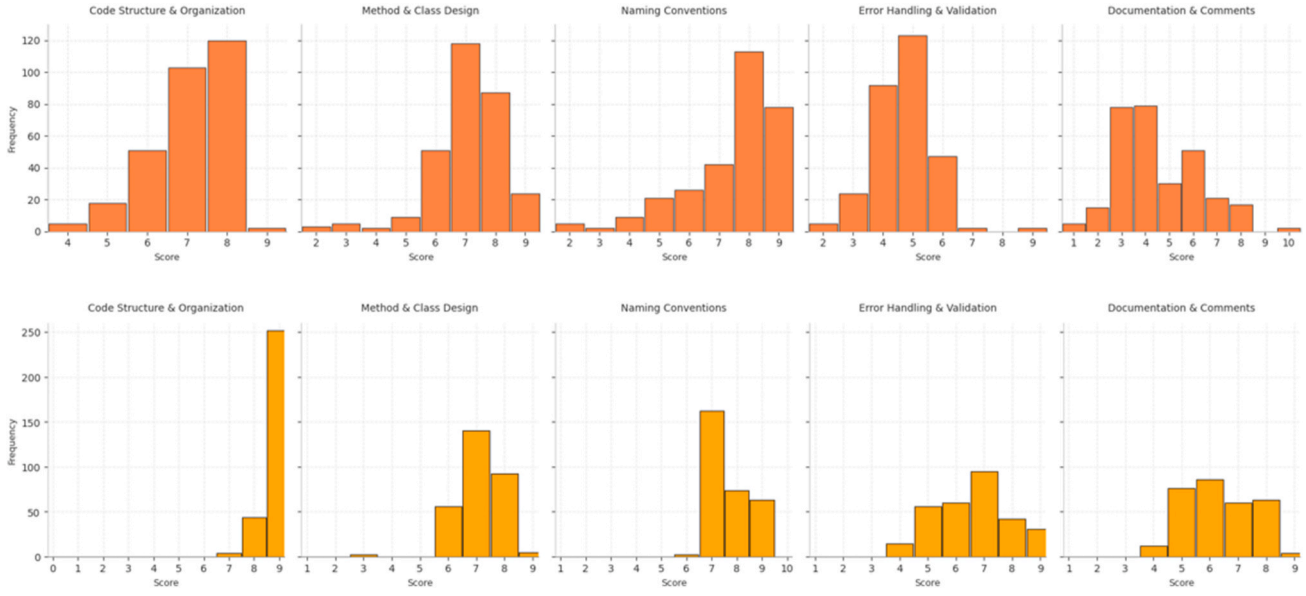



**Fig. 3.** Java code quality score distribution (ChatGPT-o4 vs DeepSeek-R1).

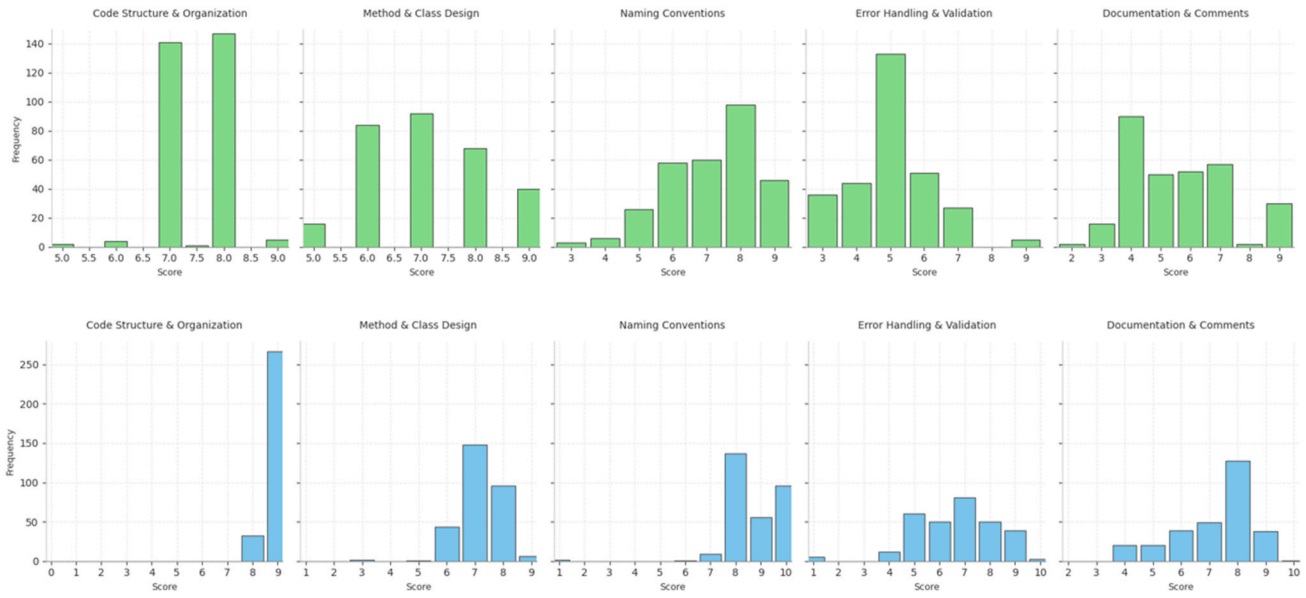



**Fig. 4.** Python code quality score distribution (ChatGPT-o4 vs DeepSeek-R1).

narrowly centered distribution, especially in Java, where most scores fall between 7 and 8. This reflects its consistent application of object-oriented principles such as encapsulation and abstraction. In contrast, ChatGPT-4o shows a broader distribution with a slight skew toward lower values. Java scores from ChatGPT-4o frequently fall below 6, indicating weaker design practices. While the Python results for ChatGPT-4o are more clustered, they still lack the refined consistency evident in DeepSeek-R1's outputs.

The naming conventions metric is the only area where ChatGPT-4o performs competitively, particularly in Python. Its scores are well-clustered around 7 to 9, with fewer low outliers. This suggests that ChatGPT-4o effectively generates human-like, semantically meaningful identifiers, which enhance code readability and clarity. However, DeepSeek-R1 also demonstrates strength in naming, particularly in Java, where its scores are tightly concentrated around 8 to 9, reflecting a disciplined approach to naming practices.

In the area of error handling and validation, DeepSeek-R1 shows a clear advantage. Its score distribution for both Python and Java is concentrated in the upper-middle range (6 to 8), indicating more consistent use of exception handling, input validation, and defensive programming practices. ChatGPT-4o, by contrast, shows a wider and lower spread. In Java, many scores fall between 3 and 5, reflecting insufficient validation and a weaker handling of edge cases.

Documentation and commenting remain among the most variable and underperforming aspects for both models, particularly for ChatGPT-4o. Its scores are broadly dispersed and skewed toward the lower end, especially in Java, where many fall between 2 and 5. DeepSeek-R1 performs better in this category, especially in Python, where a more balanced and moderately high distribution is observed. These results suggest that while documentation is not a strong suit for either model, DeepSeek-R1 offers greater consistency and completeness in explanatory comments.

Fig. 5 presents the comparative score variability analysis for Java code across the five evaluation metrics. DeepSeek-R1 demonstrates significantly greater consistency, particularly in code structure and organization, where it has a narrow interquartile range and a median near 9. This suggests that its outputs consistently adhere to fundamental structural principles. In contrast, ChatGPT-4o exhibits wider variability and more frequent low outliers, indicating occasional lapses in code architecture and overall quality.

In method and class design, DeepSeek-R1 demonstrates a clear advantage. Its score distribution is tightly clustered around higher values, reflecting a strong and consistent application of object-oriented programming principles. Conversely, the results of ChatGPT-4o are more widely distributed and with a few outliers on the low end, hypothesizing that encapsulation inconsistency, assignment of responsibility to classes, or method modularity is not as consistent. Both of the models show fairly good performance in terms of naming

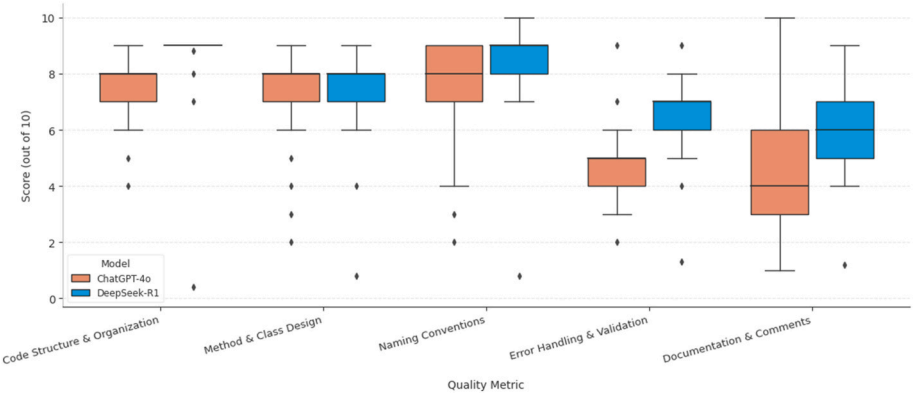



**Fig. 5.** Score variability for Java code quality metrics (ChatGPT-o4 vs DeepSeek-R1).

conventions, but DeepSeek-R1 shows a little higher median and fewer outliers, which shows that it generates identifiers in a more consistent fashion that is semantically adequate and meaningful. Compared with ChatGPT-4o, the overall score is wider, which suggests the lower consistency of adoption of existing naming standards. The occurrence of a specified gap in performance is especially evident in the error handling and validation category. DeepSeek-R1 is reliable and persistent in the use of exception handling and input validation reasoning, as depicted by high median values and constrained distribution. Conversely, the performance of ChatGPT-4o is very poor. Its distribution has several low-scoring outliers, which means that the produced code does not have enough measures to be able to handle anomalies at run time.

Documentation and comments metric adds another difference to the two models. DeepSeek-R1 also exhibits better and more stable quality documentation practices with its medium score probability distribution clustering around 6 to 7. ChatGPT-4o is very inconsistent, however, as its interquartile range is wider, and there are relatively many low outliers. It implies the under-documentation trend and the following disparity concerning the obtained comment depth and conciseness.

Interestingly, DeepSeek-R1 recorded excellent values on median scores and suppressions on interquartile ranges on all measures, suggesting a more stable and sound code generation trend than ChatGPT-4o, according to Fig. 6. Similarly, in the code structure and organization metrics, DeepSeek-R1 has more concentrated scores above zero, indicating its efficacy in providing well-structured and logically consistent Python programs. ChatGPT-4o, on the other hand, shows more variability, a smaller median, and several outliers, which are indicators of some control flow, modularity, or logical weakening.

Performance in method and class design is relatively comparable between the two models. However, DeepSeek-R1 maintains a slightly narrower score distribution, indicating more uniform application of object-oriented principles. While ChatGPT-4o performs competitively, its greater variability reflects some inconsistency in how methods and classes are structured or abstracted. In the area of naming conventions, DeepSeek-R1 outperforms ChatGPT-4o. It demonstrates both higher median scores and a notably tighter distribution, reflecting consistent use of clear, semantically appropriate identifiers.

In contrast, ChatGPT-4o′s scores are more widely dispersed, with several low-end outliers, indicating variability in naming patterns and occasional lapses in clarity. This divergence becomes the sharpest regarding error handling and validation. DeepSeek-R1 achieves the highest score most of the time, except for handling and input validation being integrated well. ChatGPT-4o, however, does poorly in this category and creates not very robust code, which is not very robust in case of uncertainties at runtime.

Once more in documentation and comments, DeepSeek-R1 performs best by keeping its comments informative and well organized. ChatGPT-4o is the one that demonstrates the most different results in this dimension; the score varies between very poor and satisfactory. Such a discrepancy decreases the readability and maintainability of its output.

Comparison of code quality metrics with the use of box plot analysis implemented on both the Java and Python programs shows that there is a deep disparity in the performance profile in the two models. In both languages, DeepSeek-R1 consistently results in higher median scores and smaller interquartile ranges and, therefore, an overall performance and consistency in major software engineering areas like control structure, method/class designs, error recovery, and documentation. Conversely, ChatGPT-4o presents more diverse scores as well as increased numbers of low outliers, especially in system error handling and documentation, which are more critical in the development of stable, sustainable code. Though ChatGPT-4o has a relative advantage in the areas of naming conventions and method design, the results of its expression are less stable and predictable, which limits its suitability for production-oriented workflows without additional correctness testing, review, and validation. Comparatively, DeepSeek-R1 has a more stable and reliable quality profile in terms of Java and Python, which makes it appropriate to be applied in the environment where maintainability, reliability and engineering rigor are central.

### 4.3. Correlation analysis of code quality factors

A correlation matrix of the factors of Java code quality created by ChatGPT-4o and DeepSeek-R1 explains significant differences in intervening adjustment of dimensions of software engineering. The greatest correlation by ChatGPT-4o is that between code structure and organization, and documentation and comments, which indicates that sometimes a simpler organization of code suggests a higher quality of inline documentation. A moderate correlation is also observed between error handling and validation and documentation, and comments, indicating some relationship between code robustness and explanatory clarity. However, naming conventions show notably weak correlations with the other factors, particularly with documentation, highlighting inconsistencies in how semantic clarity is applied to different aspects of code quality.

In contrast, the correlation matrix for DeepSeek-R1 reflects a more balanced and moderately cohesive structure. The strongest correlations appear between code structure and organization, and method and class design, as well as between method and class design and both naming conventions and documentation and comments. This pattern suggests that DeepSeek-R1 maintains moderate consistency across structural, semantic, and explanatory dimensions. While none of the correlations are especially strong, the values are relatively uniform and indicate a more integrated approach to code quality than that of ChatGPT-4o. The full correlation analysis and heatmap are presented in Fig. 7.

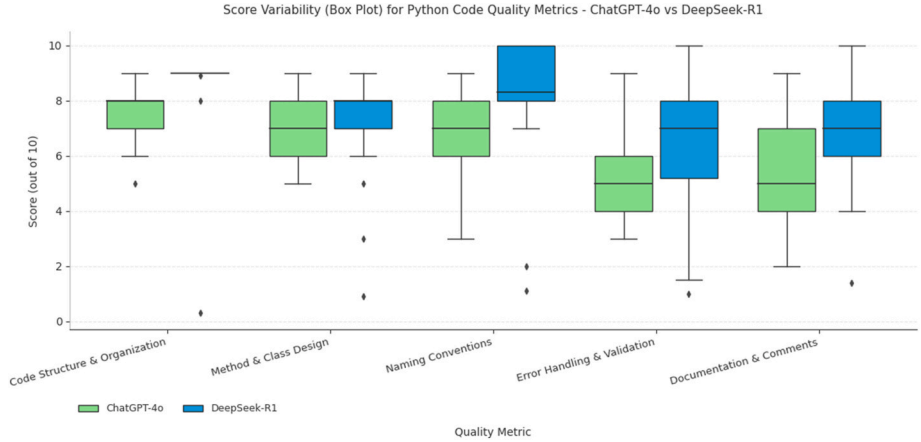



**Fig. 6.** Score variability (box plot) for Python code quality metrics (ChatGPT-o4 vs DeepSeek-R1).

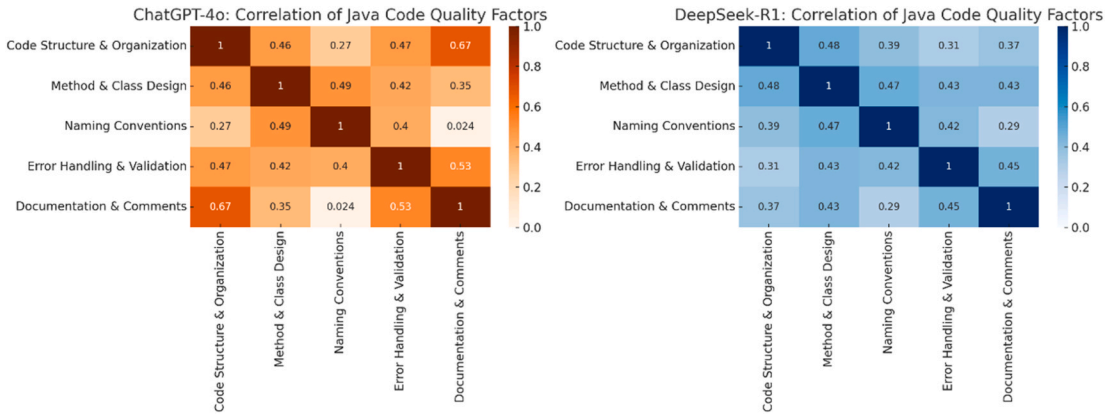



**Fig. 7.** Correlation analysis of Java code quality (ChatGPT-o4 vs DeepSeek-R1).

The correlation analysis of Python code quality factors, illustrated in Fig. 8, further highlights the differences in internal design coherence between ChatGPT-4o and DeepSeek-R1. For ChatGPT-4o, most quality factors show only weak correlations, with the strongest relationship observed between method and class design and naming conventions. Crucial elements such as error handling and documentation exhibit minimal or no significant alignment with structural or design-related factors, revealing a fragmented approach to code quality. These findings suggest that while ChatGPT-4o is capable of generating readable code in specific areas, it lacks a systematic and integrated application of holistic software engineering principles.

In contrast, DeepSeek-R1 demonstrates moderate interdependencies among several quality dimensions. The strongest observed correlation is between method and class design and naming conventions, suggesting a reliable alignment between sound design practices and clear, semantically meaningful identifiers. Additionally, moderate correlations between error handling and both documentation and method design indicate that DeepSeek-R1 more consistently integrates robustness and explanatory clarity into its code. While these relationships do not reach strong thresholds, they reflect a more cohesive and structured pattern of code generation compared to ChatGPT-4o, making DeepSeek-R1 better suited for applications where maintainability and systematic quality are critical.

These correlation patterns have practical implications for software development. For instance, the stronger relationship between method/class design and naming conventions observed in DeepSeek-R1 indicates that when a method is structurally well-designed, it is also consistently and descriptively named. In real development environments, this reduces the cognitive load on developers, as the structure and purpose of a method can be inferred directly from its name without relying heavily on documentation. Higher alignment between naming, structure, and documentation also minimizes misunderstandings during code integration, and lowers the risk of logic errors introduced during maintenance. On the other hand, weaker correlations can lead to inconsistencies where a method is well-structured but poorly named or documented, forcing developers to spend more effort interpreting the code. Thus, these correlation patterns meaningfully reflect the degree to which a model generates code that is coherent, and maintainable in real-world contexts.

Overall, the correlation patterns indicate that ChatGPT-4o's Java outputs exhibit isolated strengths, particularly in documentation when structural clarity is present, but lack consistent alignment across all quality metrics. In contrast, DeepSeek-R1 displays a more stable and cohesive profile, supporting its tendency to generate more uniformly engineered Java code, even if the inter-metric correlations remain moderate. This discussion confirms the finding that DeepSeek-R1 offers more consistent and production-ready Python code, but the outputs of ChatGPT-4o are more erratic and modular and thus can be used best in less serious programming situations.

### 4.4. Statistical performance metrics analysis

A comprehensive statistical analysis was done to compare the Java code quality created by ChatGPT-4o and the one made by DeepSeek-R1 in five major quality dimensions. It involved estimation of the failure rates, determination of variability and consistency, and a regression-derived weighting of the impact of the individual factor on the total score of the code quality. Failure rate analysis, which can be understood as the percentage of scores that are below the threshold value of 5, demonstrated that ChatGPT-4o has much higher failure scores in error

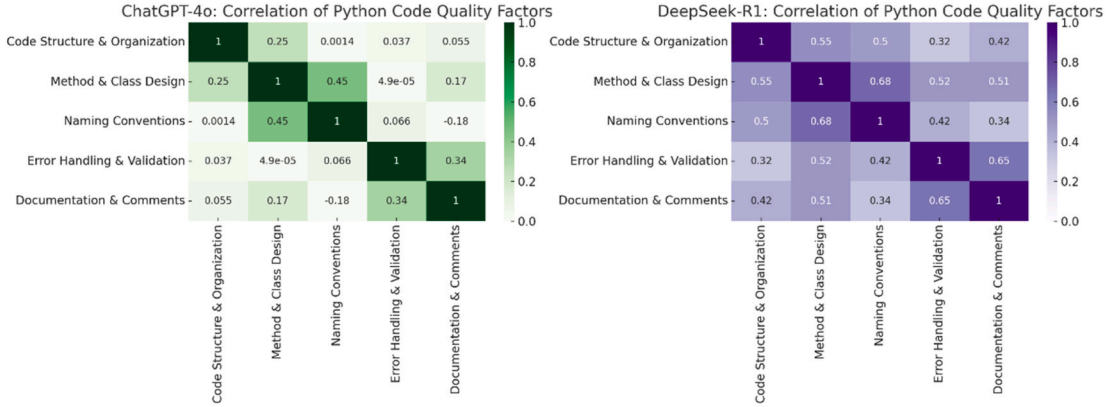



**Fig. 8.** Correlation analysis of Python code quality (ChatGPT-o4 vs DeepSeek-R1).

handling and validation and documentation and comments. Conversely, DeepSeek-R1 performed much better concerning failures on each measure, not exceeding 5 percent on error handling and 0-4 percent on the others. These findings point to DeepSeek-R1 creating more accurate and baseline Java code, particularly in locations where ChatGPT-4o shows a high degree of performance Variations.

It measured standard deviation and it demonstrated that the output of ChatGPT-4o varied more by quality dimension almost everywhere. Comparatively, DeepSeek-R1 gave quite stable outputs, especially in the code structure, which is its strength in providing constant and predictable results. DeepSeek-R1 outperformed ChatGPT-4o in consistency, computed as a consistency index when the standard deviation was inverted.

To summarize the contribution of each dimension to the overall code quality score, the study used an equal-weighted aggregation scheme. The overall quality score was calculated as the arithmetic mean of the five final dimension-level scores. Therefore, each of the five dimensions contributed an aggregation weight of 0.20 by design. These values are not interpreted as independent regression coefficients. The comparative analysis therefore focuses on the observed dimension-level means, standard deviations, failure rates, score distributions, and correlation patterns as presented in Table 11.

Similarly, the statistical evaluation of Python code generated by ChatGPT-4o and DeepSeek-R1 reveals substantial differences in reliability, variability, and the influence of quality factors on overall code scores. Failure rate analysis reveals the fact that DeepSeek-R1 outweighs the rest at every level. Although ChatGPT-4o had a much higher than average failure rate in error handling and validation, and documentation and comments, DeepSeek-R1 never exceeded 8% and has no failures in code structure, method and class design, and naming conventions. Such findings are essential debating evidence on the productivity of DeepSeek-R1 in consistently presenting Python code that is both above par in regions where ChatGPT-4o is vulnerable.

Regarding the standard deviation, the outputs of ChatGPT-4o are more variable in all of the dimensions, especially documentation and naming conventions. On the other hand, DeepSeek-R1 generates fewer random results, especially in code structure and method design. This tendency can also be proven by the consistency index, as the score of DeepSeek-R1, which measures the code structure quality, is far higher than ChatGPT-4o, which means DeepSeek-R1 has more stability and standardization.

The two models differ further in the regression analysis that provides the weight of each quality factor in predicting the overall code score. ChatGPT-4o lays the same weight of 0.20 on each of the five dimensions, implying that their influence profile is flat as well, presumably the result of scoring between the dimensions in the same way or doing so with equal penalties at the optimization. DeepSeek-R1, in its turn, shows a more sophisticated impact profile. The weights that are given to the top are method and class design, and documentation, next are the naming conventions, and finally is error handling. Interestingly, code structure has a slightly negative weight (−0.09), which may be a function of a ceiling effect, by which all but the most homogeneous structure scores may have little additional predictive value to overall translation. The above data on statistics is represented in Table 12.

All in all, these statistical comparisons demonstrate that DeepSeek-R1 completely surpasses ChatGPT-4o when it comes to producing both Java and Python code in major software engineering quality aspects. The resulting outputs of DeepSeek-R1 are of much better quality, which is more reliable, has lower failure rates, and decreased score variance and especially scores well in documentation and design consistency and semantic clearness. On the other hand, ChatGPT-4o shows progress concerning method design and naming conventions but requires more stability and quality in output as well as increased engineering-related competency in error control and document generation-based results. The regression models confirm once again these distinctions: DeepSeek-R1 has a more substantive relation of its quality scores with semantic and support-driven factors, whereas ChatGPT-4o is more analogous to a flatter, non-discriminating impact profile.

### 4.5. Functional correctness results

Table 13 summarizes the functional correctness results for the executable subset of tasks. The results show that structural quality and functional correctness are related but not identical. Some generated solutions achieved high maintainability-oriented scores but failed one or more test cases, confirming that static quality assessment alone is insufficient for determining production readiness. Therefore, the functional correctness results are interpreted alongside the five code-quality dimensions rather than replacing them.

DeepSeek-R1 achieved higher compilation/execution success rates, higher test-case pass rates, and higher full-problem correctness rates than ChatGPT-4o in both Java and Python. However, the results also confirm that functional correctness and structural quality are not identical. Some generated solutions that received acceptable maintainability-oriented scores still failed one or more test cases. Therefore, the findings should be interpreted as evidence of stronger maintainability-oriented and functional performance for DeepSeek-R1, but not as evidence that generated code is automatically production-ready. To further characterize the observed weaknesses, error and quality problems were grouped into major categories as highlighted in Table 14. This distribution helps distinguish between functional failures and maintainability-oriented weaknesses.

## 5. Discussion

### 5.1. Interpretation of results: strengths and weaknesses of ChatGPT-4o and DeepSeek-R1

The comparative assessment of the quality of code produced by ChatGPT-4o and DeepSeek-R1 using the Java and Python languages demonstrates that these models have very distinct performance patterns. In each of the evaluation dimensions, DeepSeek-R1 performs better in median scores, with less variance and fewer failures. These can be seen in the box plot distributions, failure rate analysis and the standard deviation.

In the use of Java code generation, DeepSeek-R1 has shown rigorous performance in the code structure and documentation, where there are narrow ranges in the interquartiles, and this shows high quality of results, consistency, and repeatability. Its failure rates have been insignificant on all measures, pointing to its reliability in terms of minimum quality provisions. ChatGPT-4o, conversely, performed weakly in robustness-centered domains, including error treatment and

**Table 11**
Statistical performance metrics analysis of Java codes (ChatGPT-o4 vs DeepSeek-R1).

| | ChatGPT-o4 | | | | DeepSeek-R1 | | | |
|---|---|---|---|---|---|---|---|---|
| | Failure Rate | STD | Consistency | Aggregation Weight | Failure Rate | STD | Consistency | Aggregation Weight |
| Code Structure & Organization | 0.02 | 1.01 | 0.99 | 0.20 | 0.00 | 0.40 | 2.49 | 0.08 |
| Method & Class Design | 0.04 | 1.25 | 0.80 | 0.20 | 0.01 | 0.83 | 1.20 | 0.17 |
| Naming Conventions | 0.06 | 1.59 | 0.63 | 0.20 | 0.00 | 0.82 | 1.22 | 0.19 |
| Error Handling & Validation | 0.41 | 1.04 | 0.96 | 0.20 | 0.05 | 1.34 | 0.74 | 0.17 |
| Documentation & Comments | 0.59 | 1.76 | 0.57 | 0.20 | 0.04 | 1.23 | 0.81 | 0.23 |

**Table 12**
Statistical performance metrics analysis of Python codes (ChatGPT-o4 vs DeepSeek-R1).

| | ChatGPT-o4 | | | | DeepSeek-R1 | | | |
|---|---|---|---|---|---|---|---|---|
| | Failure Rate | STD | Consistency | Aggregation Weight | Failure Rate | STD | Consistency | Aggregation Weight |
| Code Structure & Organization | 0.00 | 0.61 | 1.64 | 0.20 | 0.00 | 0.31 | 3.19 | −0.09 |
| Method & Class Design | 0.00 | 1.12 | 0.90 | 0.20 | 0.01 | 0.85 | 1.17 | 0.25 |
| Naming Conventions | 0.04 | 1.40 | 0.71 | 0.20 | 0.01 | 1.15 | 0.87 | 0.24 |
| Error Handling & Validation | 0.27 | 1.22 | 0.82 | 0.20 | 0.05 | 1.55 | 0.65 | 0.18 |
| Documentation & Comments | 0.36 | 1.71 | 0.59 | 0.20 | 0.07 | 1.42 | 0.70 | 0.25 |

**Table 13**
Functional correctness results for executable tasks.

| Language | Model | Executable tasks tested | Compilation/ execution success rate | Test-case pass rate | Full problem correctness rate |
|---|---|---|---|---|---|
| Java | ChatGPT-4o | 450 | 89.8% | 78.6% | 68.4% |
| Java | DeepSeek-R1 | 450 | 94.2% | 86.9% | 77.8% |
| Python | ChatGPT-4o | 450 | 92.4% | 82.7% | 72.9% |
| Python | DeepSeek-R1 | 450 | 96.0% | 89.8% | 81.6% |

**Table 14**
Error and quality issue distribution by model.

| Error/Issue Category | Description | Evaluation Source | ChatGPT-4o (%) | DeepSeek-R1 (%) |
|---|---|---|---|---|
| Compilation/ syntax failure | Code could not compile/execute successfully | Functional correctness testing | 8.9% | 4.9% |
| Runtime error | Code compiled/ executed but failed during runtime | Functional correctness testing | 5.6% | 3.0% |
| Incorrect output | Code executed but failed one/more test cases | Functional correctness testing | 18.5% | 11.0% |
| Weak input validation | Missing/ incomplete handling of invalid inputs/ edge cases | Static analysis + manual review | 42.8% | 23.6% |
| Weak exception handling | Missing, broad, empty, or inappropriate exception handling | Static analysis + manual review | 46.2% | 25.4% |
| Structural/ design weakness | Poor modularity, high complexity, weak cohesion, or poor separation of responsibilities | Static analysis + manual review | 31.7% | 14.8% |
| Naming inconsistency | Non-descriptive/ inconsistent identifiers | Static analysis + manual review | 18.9% | 24.3% |
| Documentation weakness | Missing, insufficient, or unclear comments/ docstrings | Static analysis + manual review | 48.5% | 29.6% |

documenting, although it exhibited independent strengths in naming conventions and procedure design. Such deficiencies were indicative of high failure rates and wide dispersion of scores, which were indicative of inconsistency in output.

The DeepSeek-R1 is once more performing well in the Python code environment with good average scores, and the results are similar in all the quality dimensions. The regression analysis shows that the maximum impact on overall quality is connected with the method design and documentation, which the DeepSeek-R1 also performed excellently. Where ChatGPT-4o did comparatively well was in the naming structures and system design, but it demonstrated low reliability and variability in error and documentation-the key elements to maintainable, production-level code. Besides, the two models are also differentiated by the results of the regression modeling. ChatGPT-4o applied the same weight to all five quality dimensions, which indicates the tendency to score based on uniformity or the fact that the system was not optimized in consideration of different priorities existing within software engineering. DeepSeek-R1, in contrast, had a robust weighting profile in the sense that more attention was given to semantically significant dimensions, e.g., documentation, naming conventions and method design. Such distribution mirrors a more realistic model in the requirements of true software development.

A discrepancy is noted in the naming conventions metric, where ChatGPT-4o consistently outperforms DeepSeek-R1 in both Java and Python. However, this discrepancy can be explained by the differing strengths of the models. ChatGPT-4o is advanced in natural language processing, which tends to generate meaningful and appropriate identifiers, that enhances code readability and aligns well with human expectations. While, DeepSeek-R1 appears to prioritize structural integrity, modularity, and fault tolerance, but may fall back on more generic or template-driven naming patterns that are less descriptive. This trade-off confirms that while DeepSeek-R1 is more reliable for engineering tasks demanding consistency and robustness, ChatGPT-4o′s advantage in naming arises from its linguistic fluency rather than an understanding of software readability. Thus, strong naming does not always correlate with stronger overall software engineering quality in AI-generated codes.

### 5.2. Impact of findings on real-world software development

These are important findings regarding their practical implications. In production systems, the quality of code directly affects maintainability, scalability and technical debt over time. To create robust software systems, the capability of producing well-documented, correctly structured, and error-tolerant code has to be in place. According to the analysis, DeepSeek-R1 is a more promising candidate for further evaluation in enterprise-scale software development workflows, particularly where maintainability, traceability, and structured error handling are important.

In comparison, the performance profile of ChatGPT-4o can be more well-suited to light applications, like educational applications, natural language-based code exploration or rapid prototyping. ChatGPT-4o does well in such situations when such generation of syntactically valid and generally readable blocks of code in response to a verbal input is the key objective, such as naming conventions and method implementation.

Nevertheless, robustness-minded dimensions have a much higher variability and failure rates, thus representing a major obstacle to the adoption of ChatGPT-4o in production quality development. Its inconsistency can make code generation less cost-competitive in regulated environments where error recovery, system reliability, or formal code review processes are critical; in these situations, the high cost of quality assurance, long debugging cycles, and disadvantages of deregulated code generation must be weighed against the advantages of more rapid

production of the initial code.

The results of this paper also justify various suggestions of how to enhance the quality of AI-produced code, especially general-purpose models such as ChatGPT-4o that are supposed to span multiple technical fields:

1. Build training data using production-quality codebases containing sophisticated error management and layered design, as well as excellent documentation. Modeling annotation of code in the industry's best practices can aid the internalization of structural reliability and applicability to the real world.
2. Includes custom post-processing layers to test and prove, clean up and comment generated code. This can involve the rule-based linters, generators of unit tests and semantic parsing-based auto-commenting to reduce inconsistency and increase output verisimilitude.
3. Training goals should be modified to focus on the less represented dimensions, like error management and documentation, by placing more weight on the variables when optimizing the training. This ensures that the model not only creates syntactic fluency but also structural and functional cognizance.
4. Optimize models to fit the domains, like production deployment, embedded systems, or a domain of education. Specialization is able to generate more situation-appropriate code production and quality expectations calibration.
5. Enable user-directed quality preferences and iterative feedback, allowing prompts like “prioritize error handling” or “include complete documentation.” Interactive refinement would enhance both the adaptability and the perceived reliability of AI-generated code.

In summary, while DeepSeek-R1 demonstrates stronger maintainability-oriented code generation in this study, deployment readiness cannot be inferred from static quality alone and requires functional correctness testing, integration testing, security assessment, and human review.

### 5.3. Chain-of-thought length and functional correctness

In this study, DeepSeek-R1 is treated as a reasoning-oriented model. However, the present evaluation is not directly measuring whether the length of the chain-of-thought reasoning is proportional to functional correctness. To ensure a fair comparison with ChatGPT-4o, any intermediate reasoning text, explanation, or <think>-style content was removed before evaluation, and only the final generated code was assessed. Therefore, functional correctness was measured from the final code artifact using compilation success, test-case pass rate, and full problem correctness, regardless the length or content of the reasoning trace. A more detailed reasoning process may help the model decompose problems, and test edge cases; however, longer reasoning is not necessarily equivalent to higher correctness. A longer chain of thought may also include irrelevant, or incorrect intermediate assumptions. Consequently, the relationship between chain-of-thought length and functional correctness should be interpreted as an open empirical question rather than a conclusion of this study.

## 6. Limitations and future work

### 6.1. Limitations

While the findings of this study provide valuable insights into the comparative quality of AI-generated code from ChatGPT-4o and DeepSeek-R1, several limitations should be acknowledged. First, the analysis was limited to two programming languages—Java and Python. Although these languages are widely used and represent both object-oriented and dynamically typed paradigms, they do not reflect the full diversity of modern programming ecosystems. Languages such as C++, Rust, Go, and JavaScript introduce different syntactic structures, memory models, and concurrency mechanisms that may pose unique challenges for AI models. Consequently, the generalizability of the findings to other language families remains uncertain.

Second, code generation was based on a controlled benchmark collection, which is, on the one hand, helpful in terms of the uniformity of the comparison but, on the other hand, not representative of the entire complexity, variability, and unpredictability of the actual software development. Such benchmark tasks were generally limited in scope and scale, and failed to consider issues such as integration of systems, multi-file systems, and relative codebase changes lowering performance that commonly occur in production systems.

Third, the analysis mainly assessed the similarity of static code quality measures, such as structure, naming, error handling and commenting, but not the dynamic behavior, like speed, precision, and the code running properly under extreme circumstances. This dynamic is an essential aspect of determining the performance of the code throughout its execution, particularly when at stress or in a high-reliability circumstance.

Fourth, although the study normalized automated tool outputs within each quality dimension, static analyzers differ in rule coverage, sensitivity, and language-specific behavior. SonarQube, PMD, and PyLint do not provide identical rule distributions across different dimensions. Therefore, cross-dimension comparisons should be interpreted with cautious. The main purpose of the automated analysis was to support consistent model-to-model comparison under the same evaluation pipeline. To mitigate this limitation, raw violation counts were not directly compared. Instead, tool findings were mapped to quality dimensions and normalized within each language–dimension context before being combined with manual rubric scores.

Another limitation concerns the absence of a formal inter-rater reliability statistic for the expert manual scores. Although the manual review was conducted by three reviewers with software engineering and programming backgrounds, and although samples were anonymized and assessed using the same predefined 0–10 rubric, the final manuscript reports consensus discussion and averaging rather than a formal statistical agreement measure. Therefore, the manual scores should be interpreted as expert-informed rubric scores rather than as scores supported by a quantified inter-rater reliability statistic.

### 6.2. Future work

Based on these research findings, a number of areas in which future research can be fruitful are suggested. First, the current analysis may be extended to a wider set of programming languages (including system-level languages, scripting languages, and domain-specific languages), to find out whether the observed trends are true of programming in general. Second, it should be tested in future research studies to analyze runtime properties of the produced code, such as memory allocated, execution speed, complexity of algorithms, and scalability. This would be more meaningful in measuring code quality, especially in applications that require high performance.

Third, putting AI-generated code into practical software development may carry some useful information on how well code is generally maintainable and integrates well in the long term. Assessment of the human code review performance of such code would contribute to filling the gap between experimental and applied studies. Also, evaluation of the development of such code by means of refactoring might be helpful. Lastly, using the input of developers, who are having to work hands-on with AI-generated code, might be useful in terms of gaining more knowledge of usability, readability, and usefulness of the code generated. A combination of statistical and user studies or pair programming simulations would provide a multidimensional view of the quality and trustworthiness of AI-made software.

## 7. Conclusion

The paper provided a detailed comparison analysis of the quality of code produced by two modern AI models, ChatGPT-4o and DeepSeek-R1, in two programming languages, Java and Python. Considering a specific range of software engineering quality measurements, it has been found that DeepSeek-R1 produced, on average, robust structurally well-defined, semantically cohesive, and fully-documented code compared to ChatGPT-4o. DeepSeek-R1 perform well in failure rates, narrower scope of score distributions and more sensible weighting of quality dimensions, meaning they fit the reality of the software industry.

On the contrary, ChatGPT-4o had strengths in both naming conventions and method structuring, but also felt inconsistent and underperformed in errors and documentation, which are notable parts of a robust and maintainable code. The deficiencies could either be identified regarding the statistical aspects or in terms of visual inspection, i.e., box plot and correlation matrices.

DeepSeek-R1 appears more suitable for development teams seeking AI-generated code with stronger maintainability-oriented properties, but production-level use still requires functional correctness testing, integration testing, security analysis, and expert review. ChatGPT-4o can be superior when it comes to educational applications, rapid prototyping, or exploring code in a natural language, privileging the freedom and speed over the engineering discipline. Developers should strike a balance between each model based on the projected needs and quality requirements.

Consistent with prior studies, our findings confirm that AI-generated code may still exhibit weaknesses in error handling, validation, and documentation. However, our results further show that the severity and consistency of these weaknesses differ substantially between ChatGPT-4o and DeepSeek-R1 and between Java and Python outputs.

Lastly, this paper explains that existing AI-assisted coding tools still require further improvement. The next versions should add semantic reasoning, refine exception flow, and create more situation-specific documentation. Moreover, recent work on hybrid frameworks that integrate generative models with either static analysis or runtime validation tools potentially can bridge performance divides. As AI keeps changing software engineering on a professional scale, it will be an important part of its controlled and meaningful use to make sure that code produced by the AI is experienced and follows professional standards.

## CRediT authorship contribution statement

**Sally Almanasra:** Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Writing – original draft. **Khaled Suwais:** Conceptualization, Methodology, Supervision, Writing – original draft. **Suad AlRamouni:** Conceptualization, Validation, Writing – review & editing.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgments

The authors Sally Almanasra and Suad AlRamouni would like to thank Prince Sultan University for paying the publication fees for this work.

The author Khaled Suwais would also like to thank the Arab Open University, Saudi Arabia for supporting this work.

## Data availability

The data are available in the paper

## References

[1] Tembhekar P, Devan M, Jeyaraman J. Role of GenAI in automated code generation within devops practices: explore how generative AI. J Knowl Learn Sci Technol 2023;2(2):500–12. https://doi.org/10.60087/Jklst.Vol2.N2.P512. ISSN 2959-6386 (Online).
[2] Shojaee P, Jain A, Tipirneni S, Reddy C. Execution-Based code generation using deep reinforcement learning. 2023. https://doi.org/10.48550/Arxiv.2301.13816.
[3] Odeh A, Odeh N, Mohammed A. A comparative review of AI techniques for automated code generation in software development: advancements, challenges, and future directions. TEM J 2024:726–39. https://doi.org/10.18421/Tem131-76.
[4] Tsai C, Taylor G. Deeprng: towards deep reinforcement learning-assisted generative testing of software. https://Doi.Org/10.48550/Arxiv.2201.12602; 2022.
[5] Uzair W, Naz S. Six-Tier architecture for Ai-Generated software development: a large language models approach. 2023. https://doi.org/10.21203/Rs.3.Rs-3086026/V1.
[6] Uyanık B, Sayar A. Developing web-based process management with automatic code generation. Appl Sci 2023;13(21):11737. https://doi.org/10.3390/App132111737.
[7] Leung M, Murphy G. On automated assistants for software development: the role of LLMS. 2023. p. 1737–41. https://doi.org/10.1109/ase56229.2023.00035.
[8] Taeb M, Chi H, Bernadin S. Assessing the effectiveness and security implications of AI code generators. J Colloquium Inf Syst Secur Educ 2024;11(1):6. https://doi.org/10.53735/cisse.v11i1.180.
[9] Cambaz D, Zhang X. Use of AI-driven code generation models in teaching and learning programming: a systematic literature review. 2024. p. 172–8. https://doi.org/10.1145/3626252.3630958.
[10] Idrisov B, Schlippe T. Program code generation with generative AIs. Algorithms 2024;17(2):62. https://doi.org/10.3390/a17020062.
[11] Tosi D. Studying the quality of source code generated by different AI generative engines: an empirical evaluation. Future Internet 2024;16(6):188. https://doi.org/10.3390/fi16060188.
[12] Majeed A, Hwang S. Towards unlocking the hidden potentials of the data-centric AI paradigm in the modern era. Appl Syst Innovat 2024;7(4):54. https://doi.org/10.3390/asi7040054.
[13] Negri-Ribalta C, Geraud-Stewart R, Sergeeva A, Lenzini G. A systematic literature review on the impact of AI models on the security of code generation. Front Big Data 2024;7:1386720.
[14] Alenezi M, Akour M. AI-driven innovations in software engineering: a review of current practices and future directions. Appl Sci 2025;15(3):1344. https://doi.org/10.3390/app15031344.
[15] Golendukhina V, Lenarduzzi V, Felderer M. What is software quality for AI engineers? Towards a thinning of the fog. In: Proceedings of the 1st Conference on AI Engineering—Software Engineering for AI (CAIN '22). Association for Computing Machinery; 2022. p. 1–9. https://doi.org/10.1145/3522664.3528599.
[16] Kharchenko V, Fesenko H, Illiashenko O. Quality models for artificial intelligence systems: Characteristic-based approach, development and application. Sensors 2022;22(13):4865. https://doi.org/10.3390/s22134865.
[17] Almanasra S, Suwais K. Analysis of ChatGPT-Generated codes across multiple programming languages. IEEE Access 2025;13:23580–96. https://doi.org/10.1109/ACCESS.2025.3538050.
[18] Liu Z, Tang Y, Luo X, Zhou Y, Zhang L. No need to lift a finger anymore? Assessing the quality of code generation by ChatGPT. IEEE Trans Software Eng 2024;50(6):1548–84. https://doi.org/10.1109/tse.2024.3392499.
[19] Abraham R. Democratizing AI's frontiers: a critical review of DeepSeek AI's open-source ecosystem. https://doi.org/10.31219/osf.io/9r7e3_v1; 2025.
[20] Coignion T, Quinton C, Rouvoy R. A performance study of LLM-generated code on LeetCode. 2024. p. 79–89. https://doi.org/10.1145/3661167.3661221.
[21] Ding Y, Min M, Kaiser G, Ray B. Cycle: learning to self-refine the code generation. Proc ACM Program Lan 2024;8:392–418. https://doi.org/10.1145/3649825. OOPSLA1.
[22] Liu Y, Tantithamthavorn C, Liu Y, Li L. On the reliability and explainability of automated code generation approaches. https://doi.org/10.48550/arxiv.2302.09587; 2023.
[23] Jamdade M, Liu Y. A pilot study on secure code generation with ChatGPT for web applications. 2024. p. 229–34. https://doi.org/10.1145/3603287.3651194.
[24] Cotroneo D, Improta C, Liguori P. Human-Written vs. AI-Generated code: a large-scale study of defects, vulnerabilities, and complexity. Proceedings of the 36th IEEE international symposium on software reliability engineering (ISSRE 2025). 2025. https://doi.org/10.48550/arXiv.2508.21634.
[25] Nguyen Van-Viet, Nguyen Huu-Khanh, Nguyen The-Vinh, Vu Duc-Quang. Phi-3-Code: fine tuning a small size Language model for coding generation. In: Dao NN, Le HA, Vadivel R, Nguyen NT, editors. Intelligence of things: technologies and applications, ICIT 2025, lecture notes on data engineering and communications technologies, vol. 282. Cham, Switzerland: Springer; 2026. p. 247–57. https://doi.org/10.1007/978-3-032-13254-3_20.
[26] Nguyen V-V, Nguyen H-K, Nguyen K-S, Luong TM-H, Vu D-Q, Phung T-N, Nguyen T-V. A novel unified framework for automated generation and multimodal

validation of UML diagrams. Comput Model Eng Sci Jan. 2026;146(1):33–60. https://doi.org/10.32604/cmes.2025.075442.
[27] LeetCode. LeetCode online judge [Online]. Available: https://leetcode.com/. [Accessed 20 January 2025].
[28] Hackerrank. Hackerrank platform [Online]. Available: https://www.hackerrank.com/. [Accessed 20 January 2025].
[29] Codeforces. Codeforces platform [Online]. Available: https://codeforces.com/. [Accessed 24 January 2025].
[30] Azhir E, Zakeri M, Abedini Y, Ghahfarokhi MM. QualCode: a data-driven framework for predicting software maintainability based on ISO/IEC 25010. Sci Comput Program 2026;250:103399. https://doi.org/10.1016/j.scico.2025.103399.
[31] Suwais K, Hnaif AA, Almanasra S. An alternative static taint analysis framework to detect PHP web shell–based web attacks. Int J Adv Soft Comput Its Appl 2023;15 (3). https://doi.org/10.15849/IJASCA.231130.08. November 2023.
[32] Marashdih AW, Zaaba ZF, Suwais K, Mohd NA. Web application security: an investigation on static analysis with other algorithms to detect cross site scripting. Procedia Comput Sci 2019;161:1173–81. https://doi.org/10.1016/j.procs.2019.11.230.
[33] Marashdih AW, Zaaba ZF, Suwais K. An enhanced static taint analysis approach to detect input validation vulnerability. J King Saud Univ, Comput Inf Sci 2023;35(2): 682–701. https://doi.org/10.1016/j.jksuci.2023.01.009.
[34] SonarQube code review, Sonar [Online]. Available, https://www.sonarsource.com/. [Accessed 12 February 2025].
[35] PyLint – code analysis for python [Online]. Available Pylint https://pylint.org/ 15-2-2025.